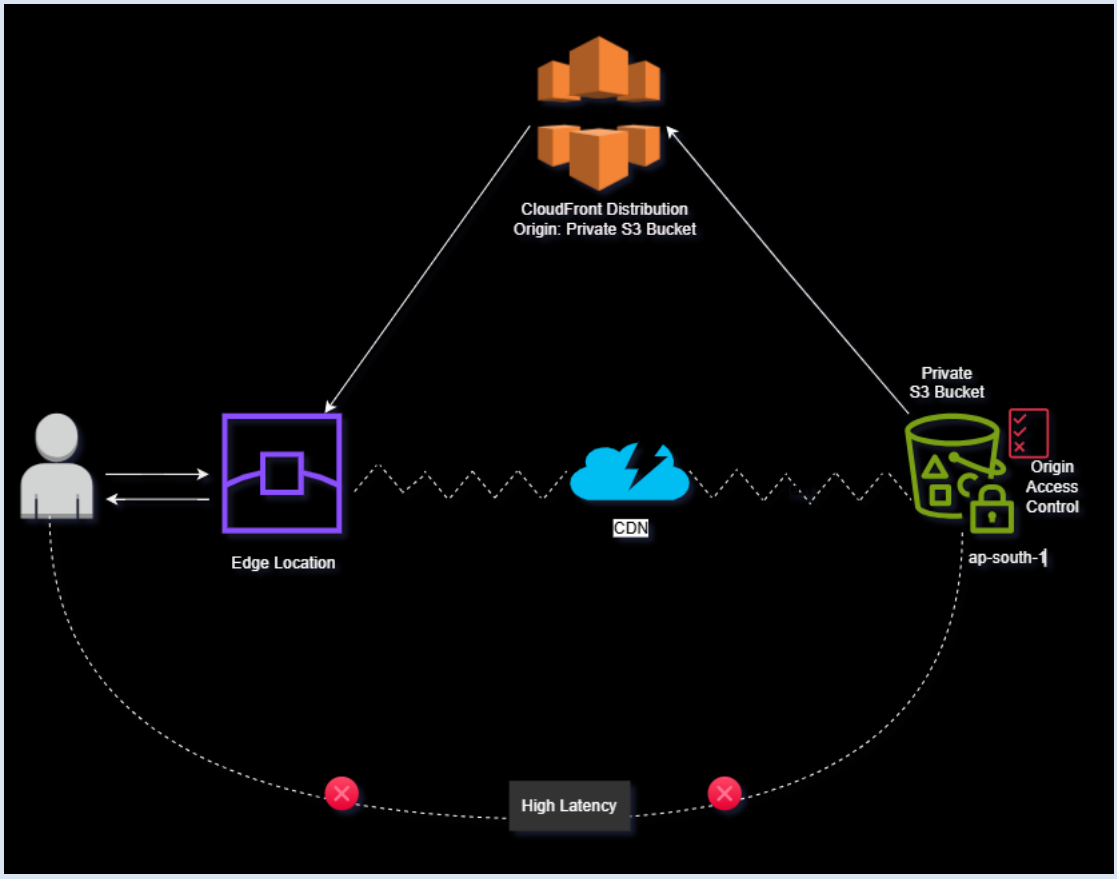
Project :- Secure Static Website Hosting using Amazon S3 and CloudFront

Designed a secure static website hosting architecture using **Amazon S3** and **Amazon CloudFront**. The solution uses a private S3 bucket with **Origin Access Control (OAC)** to prevent direct public access, while **CloudFront edge locations cache and deliver content globally**, reducing latency and improving performance.



What This Project Demonstrates

This project shows important AWS concepts:

1. CDN: - Using **Amazon CloudFront** for global content delivery.

2. Security: - Private S3 bucket.

3. OAC: - Secure communication between: CloudFront → S3

4. Caching: - Edge location caching.

5. Cache Invalidation: - Cache Invalidation: Clearing cached objects so users receive updated content.

Step 1 — Create S3 Bucket

Go to

AWS Console → S3 → Create bucket

Settings:

Bucket Name: my-secure-static-site
Region: ap-south-1 (Mumbai)

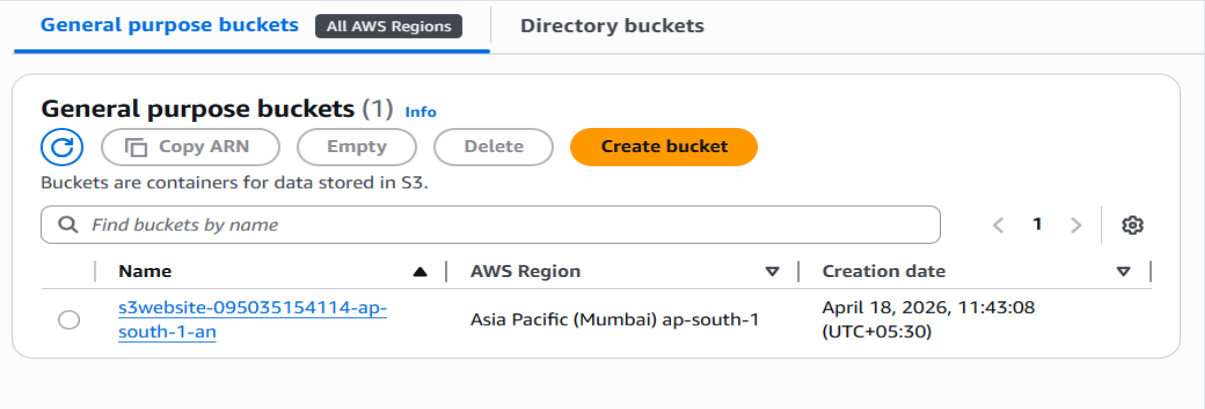
Important:

Block Public Access → ENABLED

Do NOT enable:

Static website hosting

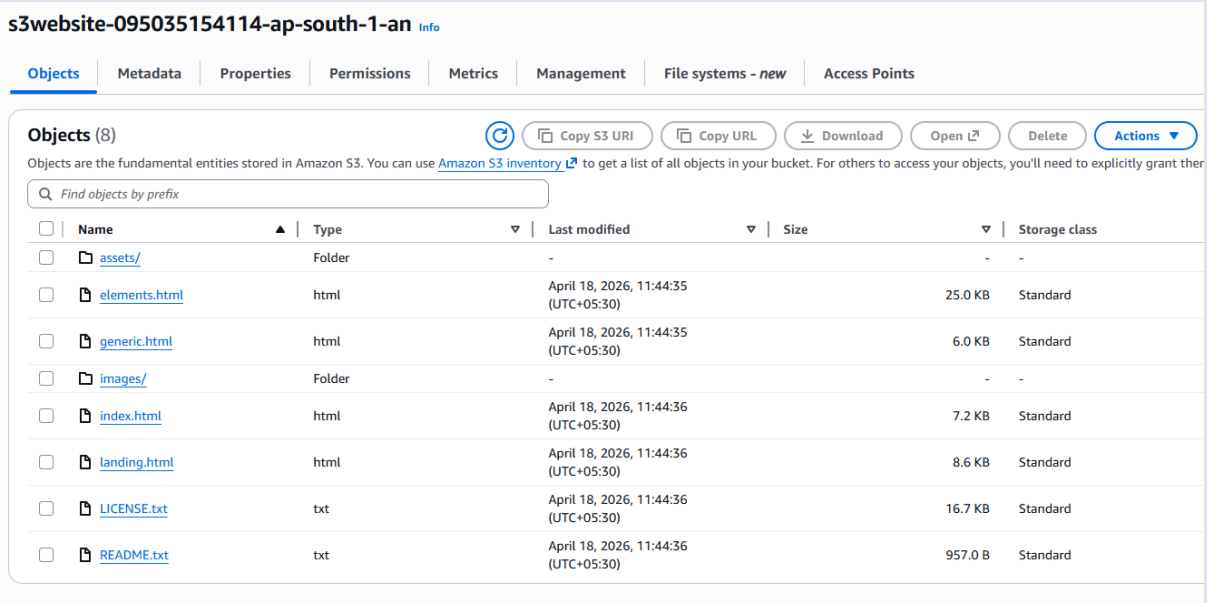
Create bucket.



Step 2 — Upload Website Files

Upload:

Upload your files



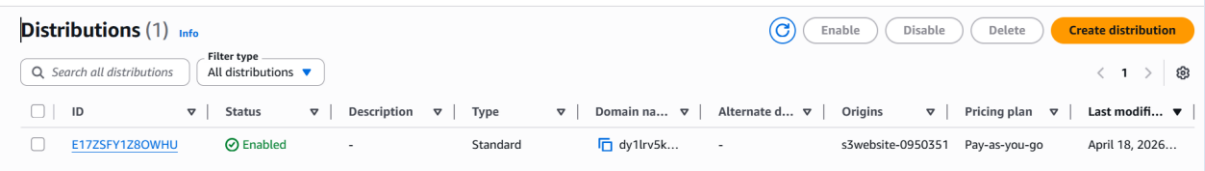
Step 3 — Create CloudFront Distribution

Go to

CloudFront → Create Distribution

Origin settings:

Origin type → S3
Origin domain → select your S3 bucket



Step 4 — Configure Origin Access

Choose:

Origin Access → Origin Access Control (OAC)

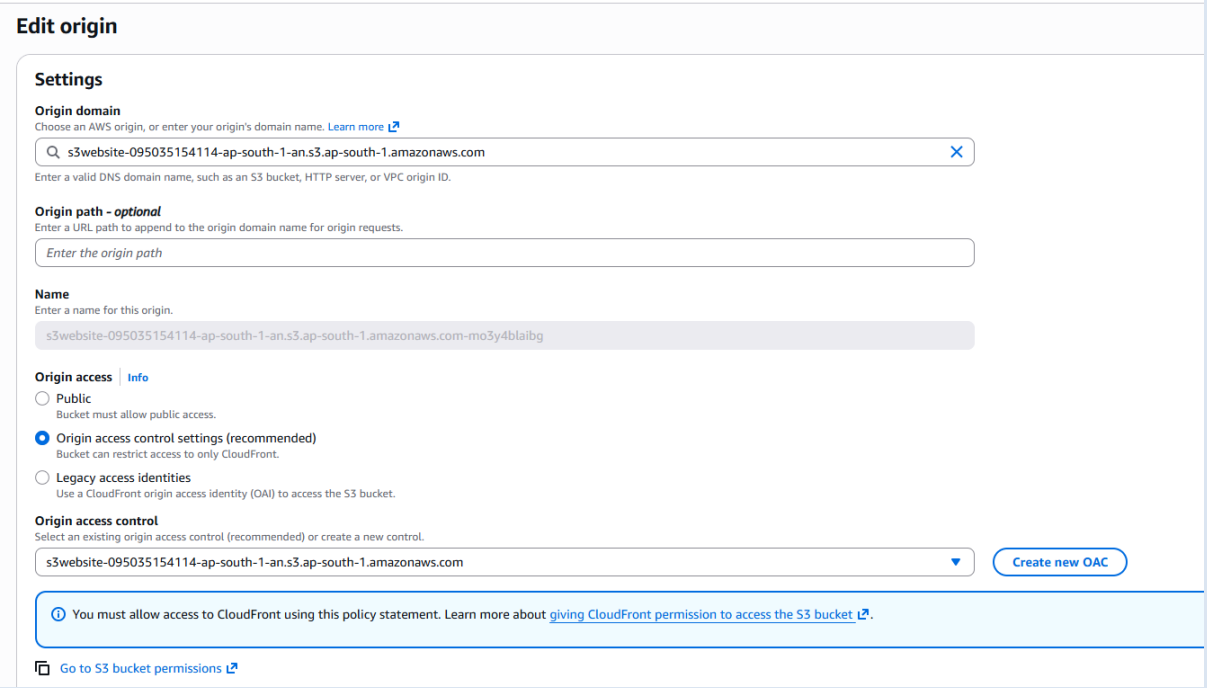
Click

Create new OAC

Settings:

Signing behavior → Sign requests
Origin type → S3

Create OAC.



Attach it to the distribution.

Step 5 — Configure Distribution Settings

Recommended settings:
Viewer Protocol Policy → Redirect HTTP to HTTPS
Cache Policy → CachingOptimized
WAF → Disabled (for this project)
Default root object → index.html

Create distribution.

Step 6 — Update S3 Bucket Policy

CloudFront will generate a **policy**.

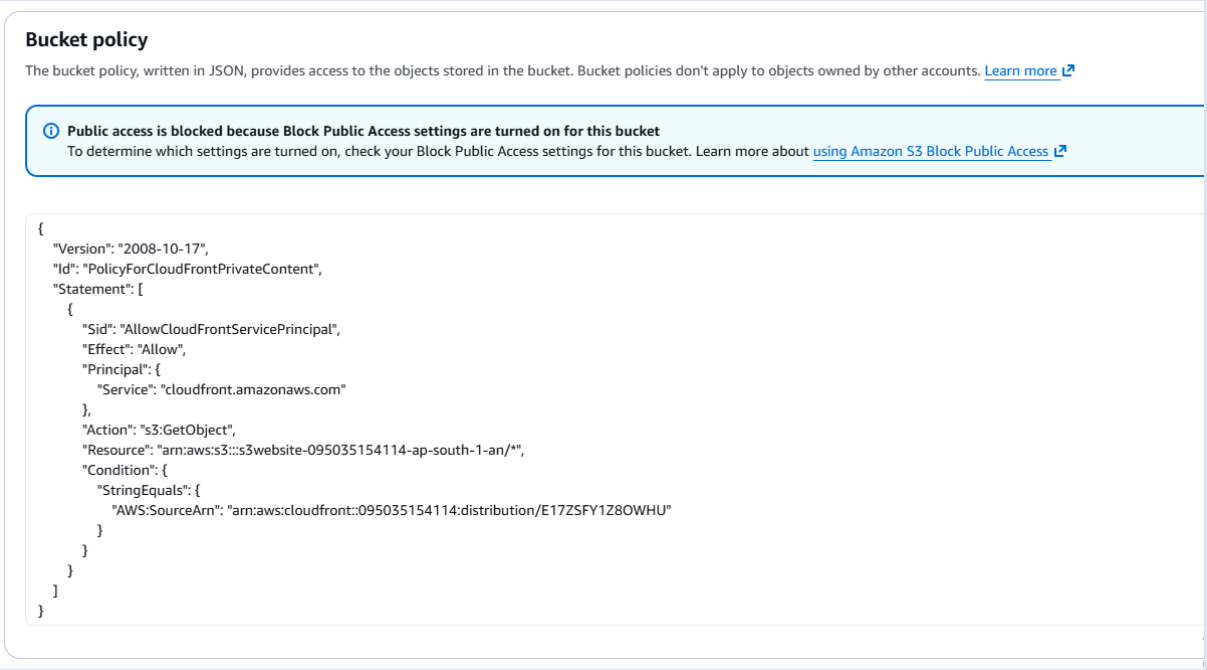
Copy it.

Go to:

S3 → Bucket → Permissions → Bucket Policy

Paste the policy.

Only CloudFront can access S3
Users cannot directly access S3



Bucket policy

The bucket policy, written in JSON, provides access to the objects stored in the bucket. Bucket policies don't apply to objects owned by other accounts. Learn more

Public access is blocked because Block Public Access settings are turned on for this bucket
To determine which settings are turned on, check your Block Public Access settings for this bucket. Learn more about using Amazon S3 Block Public Access


```
{
  "Version": "2008-10-17",
  "Id": "PolicyForCloudFrontPrivateContent",
  "Statement": [
    {
      "Sid": "AllowCloudFrontServicePrincipal",
      "Effect": "Allow",
      "Principal": {
        "Service": "cloudfront.amazonaws.com"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::s3website-095035154114-ap-south-1-an/*",
      "Condition": {
        "StringEquals": {
          "AWS:SourceArn": "arn:aws:cloudfront::095035154114:distribution/E17ZSFY1Z8OWHU"
        }
      }
    }
  ]
}
```

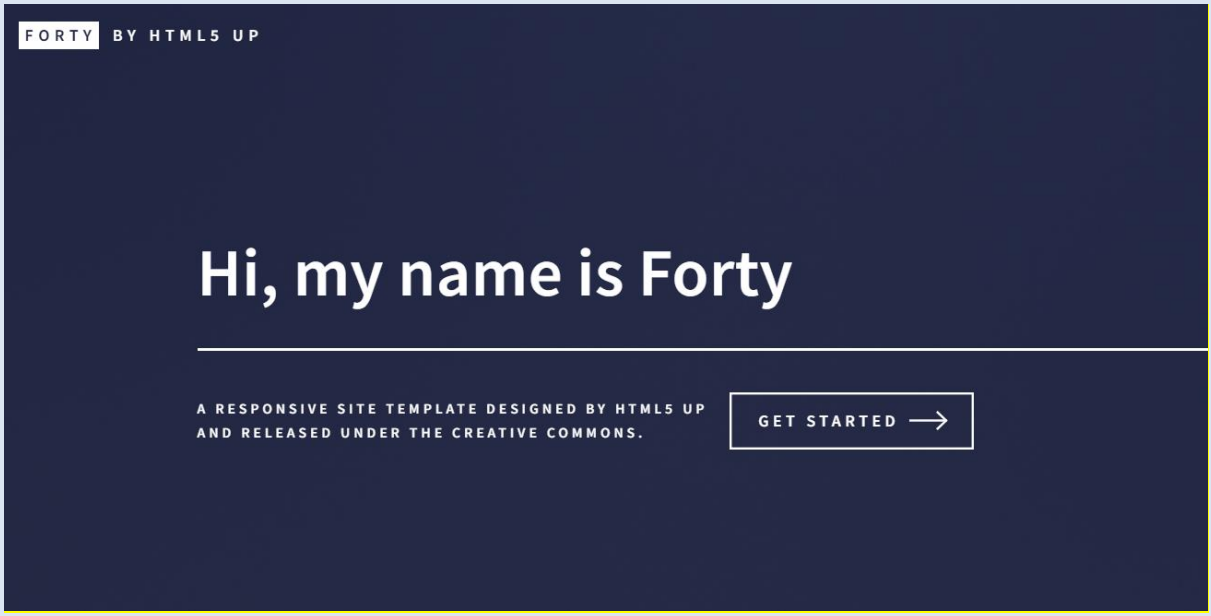
Step 7 — Access the Website

CloudFront will generate a domain :

dy1lrv5knqdhl.cloudfront.net

https://dy1lrv5knqdhl.cloudfront.net

Website loads from **Edge Locations**.



Step 8 — Modify Website Content

Now suppose you change text in:
index.html

```
<section id="banner" class="major">
        <div class="inner">
                <header class="major">
                        <h1>This is Cloud Project</h1>
                </header>
                <div class="content">
```

Upload the updated file to **S3 bucket**.

Step 9— Why Changes Don't Appear Immediately

Because **CloudFront caches files**.

Edge locations already stored the old version.

User → Edge location
Edge location → cached index.html

So users see **old content**.

Step 10 — Cache Invalidation

To update content:

Go to

CloudFront → Distribution

Click on Invalidations

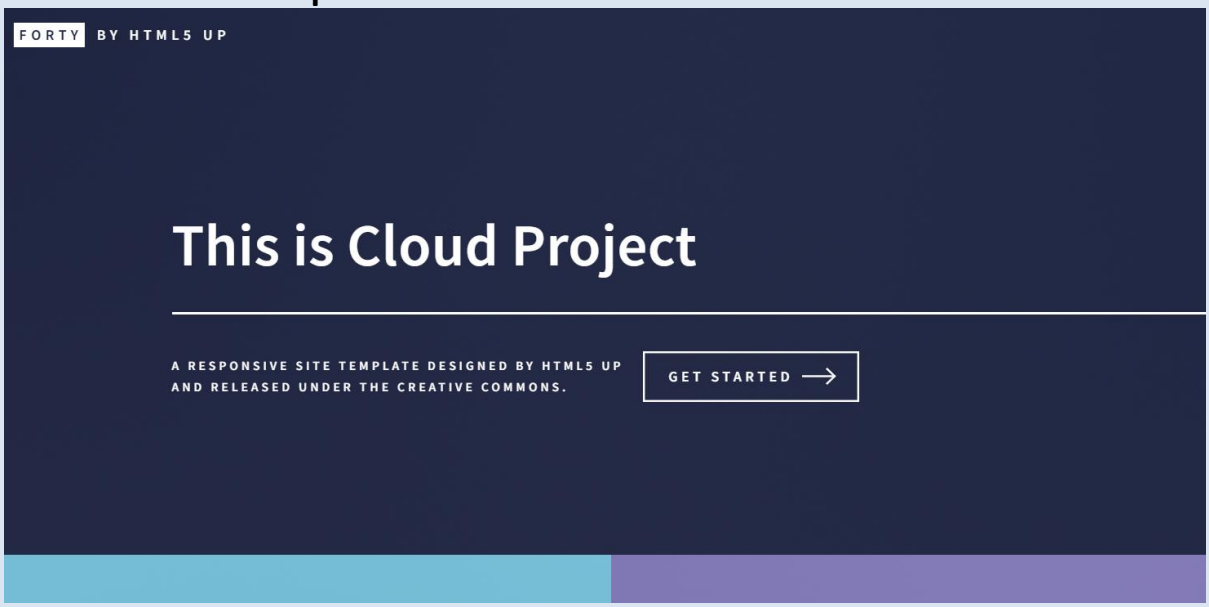
→ Create invalidation.

Path: /index.html Or invalidate everything: /*

Click: → Create Invalidation

CloudFront clears cached files.

Now users receive **updated content**.



Summary of What We Did:

- Created a private **Amazon S3** bucket to store static website files.
- Configured **Amazon CloudFront** distribution with the S3 bucket as origin.
- Enabled **Origin Access Control (OAC)** to keep the S3 bucket private.
- Routed user requests through CloudFront edge locations.
- Cached content at edge locations to reduce latency.
- Prevented direct public access to the S3 bucket.
- Achieved secure and faster global content delivery.